Listing 2.3 Loading data, casting it, measuring it, and displaying it as a bar chart

```
d3.csv("cities.csv",function(error,data) {dataViz(data);});

function dataViz(incomingData) {

 var maxPopulation = d3.max(incomingData, function(el) {
     return parseInt(el.population);}
 );
 var yScale = d3.scale.linear().domain([0,maxPopulation]).range([0,460]);
 d3.select("svg").attr("style","height: 480px; width: 600px;");
 d3.select("svg")
   .selectAll("rect")
   .data(incomingData)
   .enter()
   .append("rect")
   .attr("width", 50)
   .attr("height", function(d) {return yScale(parseInt(d.population));})
   .attr("x", function(d,i) {return i * 60;})
   .attr("y", function(d) {return 480 - yScale(parseInt(d.population));})
   .style("fill", "blue")
   .style("stroke", "red")
   .style("stroke-width", "1px")
   .style("opacity", .25);
}
```

Transforms the population value into an integer

Creating a bar chart out of the Twitter data requires a bit more transformation. As shown in the following listing, we use `d3.nest()` to gather the tweets under the person making them, and then use the length of that array to create a bar chart of the number of tweets (shown annotated in figure 2.22).

Listing 2.4 Loading, nesting, measuring, and representing data

```
d3.json("tweets.json",function(error,data) {dataViz(data.tweets)});
function dataViz(incomingData) {

var nestedTweets = d3.nest()
.key(function (el) {return el.user;})
.entries(incomingData);

nestedTweets.forEach(function (el) {
el.numTweets = el.values.length;
})

var maxTweets = d3.max(nestedTweets, function(el) {return el.numTweets;});

var yScale = d3.scale.linear().domain([0,maxTweets]).range([0,100]);

d3.select("svg")
  .selectAll("rect")
  .data(nestedTweets)
  .enter()
  .append("rect")
  .attr("width", 50)
```

Specifies data.tweets, where your data array is located

Creates a new attribute based on the number of tweets